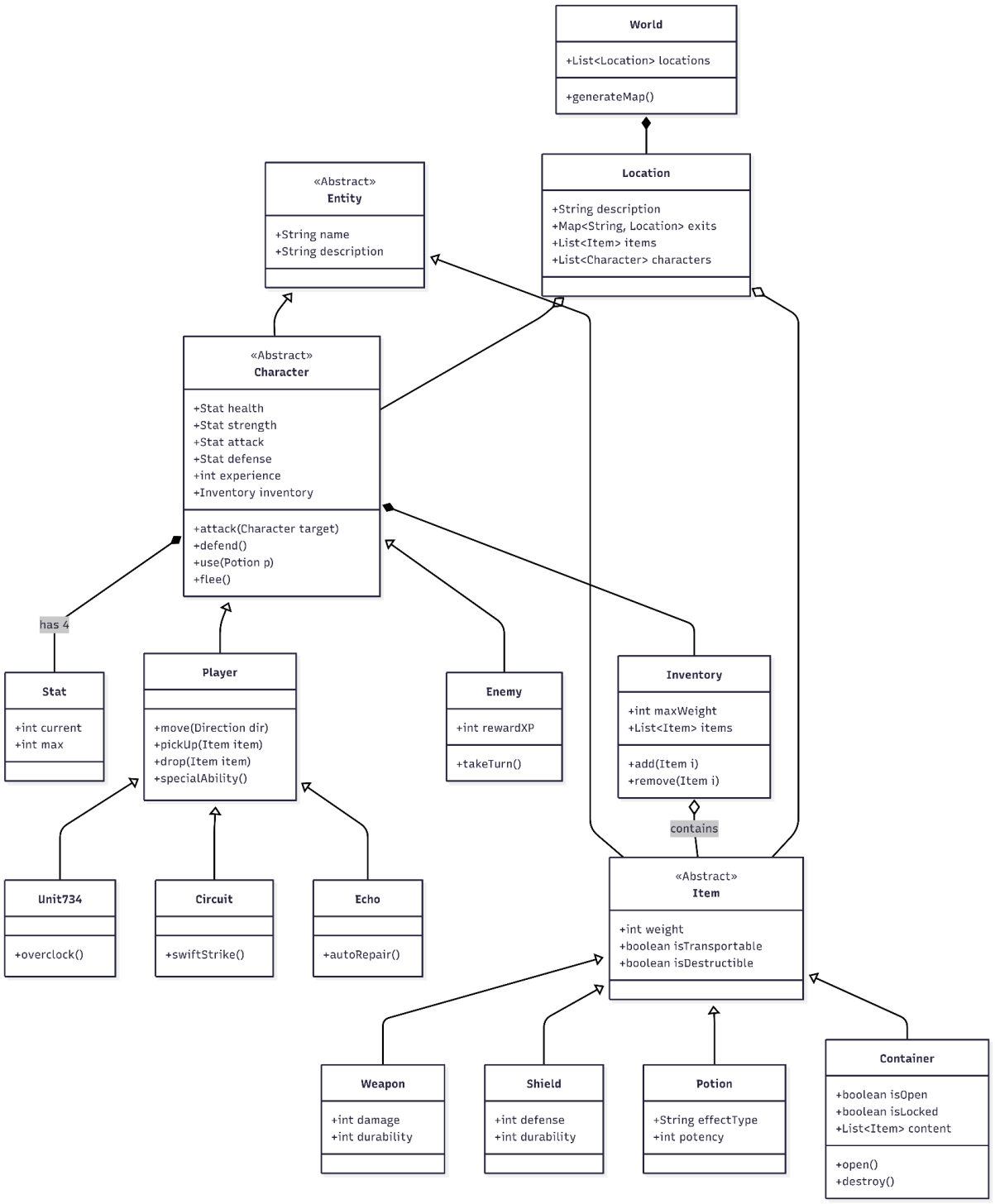
DOCUMENTAZIONE DEL PROGETTO: RPG GAME



Funzionamento generale:

Il progetto prevede lo sviluppo di un **gioco di ruolo testuale** in lingua inglese , ambientato in un mondo strutturato come un **grafo di locazioni** collegate tra loro cardinalmente (Nord, Sud, Ovest, Est).

Ecco come funzionerà tecnicamente:

- **Esplorazione e Interazione**: Ti sposterai tra luoghi descritti testualmente dove potrai trovare oggetti (armi, scudi, pozioni o contenitori) e personaggi. L'interazione avverrà tramite **comandi testuali** come look, north o pick up.
- **Gestione Personaggio**: Il protagonista possiede statistiche dinamiche (**vita, forza, attacco e difesa**) e una **sacca** con limite di peso. Sconfiggere nemici aumenta l'**esperienza**, permettendo di potenziare i parametri una volta raggiunta una determinata soglia.
- **Combattimento**: Gli scontri sono **a turni**. Potrai scegliere se attaccare, difenderti, usare pozioni o scappare, tenendo conto dell'usura delle armi e degli scudi che possono rompersi con l'uso

Storia e personaggi:

1. Introduzione e Ambientazione

Il progetto consiste in un gioco di ruolo (RPG) testuale ambientato in un futuro distopico nell'anno 2099. La storia si svolge a Neo-Veridia, una metropoli dominata dalla potente "Omni-Corp". Il giocatore interpreta un droide che ha sviluppato una propria coscienza e deve attraversare la città per recuperare il "Source Code", un file leggendario per liberare la propria specie.

2. Meccaniche di Gioco

Il sistema di gioco si basa su tre pilastri fondamentali:

- Esplorazione a Grafo: Il mondo è diviso in `Location` (luoghi) collegate tramite direzioni cardinali (North, South, East, West). Ogni luogo offre descrizioni ambientali e possibili interazioni.
- Gestione Parametri: Ogni personaggio è definito da quattro statistiche principali: Health (Vita), Strength (Forza/Carico), Attack (Attacco) e Defense (Difesa). Ogni statistica ha un valore attuale e un valore massimo.
- Sistema di Combattimento e Crescita: I combattimenti sono a turni. Sconfiggere i nemici permette di accumulare Experience (XP). Al raggiungimento di 50 XP, il personaggio sale di livello, migliorando le proprie prestazioni.

3. Classi di Personaggi Selezionabili

All'inizio della partita, l'utente potrà scegliere tra tre unità diverse, ognuna con uno stile di gioco specifico:

1. Unit 734 (The Tank): Specializzata nella resistenza. Possiede alta difesa e l'abilità *Overclock* per raddoppiare temporaneamente la forza a costo di alcuni punti vita.

2. Unit "Circuit" (The Scout): Focalizzata sulla velocità. Ha meno vita ma un attacco più alto e la possibilità di colpire due volte nello stesso turno (*Swift Strike*).
3. Unit "Echo" (The Support): Unità di manutenzione con la capacità di trasporto più elevata. Possiede l'abilità *Auto-Repair* che ripristina vita durante gli spostamenti tra le zone.

4. Elementi del Mondo (Entità)

Il mondo di gioco è popolato da oggetti e minacce:

- Items (Oggetti): Divisi in armi (`Weapon`), scudi (`Shield`) e pozioni (`Potion`). Gli oggetti hanno un peso che influisce sul limite di carico del personaggio.
- Containers (Contenitori): Casse o archivi digitali che possono essere aperti, distrutti o che richiedono una chiave (`Passcode`) per accedere al contenuto.
- Enemies (Nemici): Droni di sorveglianza, guardie corporative e boss digitali, ognuno con statistiche bilanciate per offrire diversi livelli di sfida.

LINK GITHUB

1. Pietro: Core Engine & User Interface

Pietro si occupa della "cornice" del gioco, ovvero di come il software si avvia e comunica con l'utente.

- Main & Game Loop: Gestione del ciclo principale `while(running)`.
- Command Parser: Creazione della logica che legge le stringhe dell'utente (es. "go north") e le trasforma in azioni.
- JavaFX UI (Opzionale): Se il gruppo decide per la versione grafica, Pietro creerà la finestra principale, l'area di testo e i pulsanti per le abilità.
- Package `it.school.neonshadows.engine`

2. Noe: Game Logic & World Structure

Noe si occupa del "cuore" logico, ovvero delle regole del mondo e del movimento.

- World & Location: Gestione del grafo della mappa. Deve scrivere il codice che collega la "Scrap Yard" ai "Neon Slums" tramite direzioni cardinali.
- Combat System: Implementazione dei turni di combattimento, calcolo dei danni (Attacco vs Difesa) e gestione della morte dei nemici.

- Leveling System: Codice che controlla quando la variabile `experience` raggiunge 50 e incrementa i `maxValues` delle statistiche.
- Package `it.school.neonshadows.world` e `it.school.neonshadows.logic`

3. Cristiano: Data Classes & Content

- Entity & Items: Implementazione delle classi `Weapon`, `Shield`, `Potion` e `Container`.
- Enemy Templates: Creazione delle diverse tipologie di nemici (Scavenger Drone, Corp-Enforcer) con i loro valori di statistica bilanciati.
- Text Database: Scrittura di tutte le stringhe di testo in inglese (descrizioni delle stanze, messaggi di errore, dialoghi).
- Package `it.school.neonshadows.entities`

4. Nicholas: Integration, QA & Documentation

- Integration Testing: Verifica che gli oggetti creati da Cristiano possano essere raccolti nel mondo creato da Noe e usati nel motore di Pietro.
- GitHub Management: Risoluzione dei conflitti nei commit e organizzazione delle cartelle.
- Final Documentation: Inserimento dei commenti Javadoc nel codice per spiegare cosa fa ogni metodo (essenziale per il voto di "Qualità del Codice").